

Android XML 原生TimePicker详解（基础+自定义+实战补充）

在Android开发中，`TimePicker` 是系统提供的原生时间选择控件，用于接收用户输入的“时-分”信息，支持12小时制与24小时制切换，可直接嵌入布局使用。与封装好的 `TimePickerDialog` 不同，`TimePicker` 具备更高的灵活性，开发者可根据需求自定义其视觉样式、交互逻辑与功能扩展。本文将从“原生基础用法”“核心API解析”“自定义开发”“实战优化”四个维度全面讲解，搭配可直接运行的代码示例，帮助开发者快速掌握其使用技巧。

一、核心概念：TimePicker的本质与特性

`TimePicker` 是Android.widget包下的可视化控件，核心作用是提供时间选择交互，其本质是对“时-分”选择逻辑与UI的封装，核心特性如下：

- 双时间制支持**：默认跟随系统时间制，可通过属性或代码手动设置为12小时制（含AM/PM标识）或24小时制，适配不同地区使用习惯。
- 两种布局模式**：支持“滚轮模式（spinner）”和“时钟模式（clock）”，前者通过滚轮滑动选择时间，后者模拟时钟表盘点击选择，可通过API切换。
- 实时交互回调**：通过 `OnTimeChangeListener` 监听时间变化，用户滑动或点击选择时实时触发，便于即时更新UI。
- 原生样式适配**：样式与系统主题（如Theme.AppCompat、Theme.MaterialComponents）自动同步，高版本支持通过主题属性快速调整颜色、字体等。
- 轻量扩展**：支持直接嵌入布局，可与其他控件（如Button、TextView）联动，实现复杂业务逻辑（如时间范围限制、分钟步长控制）。

 **关键区别**：`TimePicker` 是“独立控件”，需手动处理布局与显示逻辑；`TimePickerDialog` 是“对话框封装”，将 `TimePicker` 嵌入AlertDialog，无需关注弹窗逻辑，适合快速使用。

二、基础用法：原生TimePicker（3步快速实现）

原生 `TimePicker` 的使用流程为“布局中引入控件→代码中绑定配置→设置时间变化回调”，适合简单的时间选择场景（如表单中的时间输入项）。

步骤1：布局中引入TimePicker

在 `activity_main.xml` 中引入 `TimePicker`，搭配按钮和文本控件实现“选择时间→确认→显示结果”的完整流程：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:orientation="vertical"
7      android:padding="20dp"
8      android:background="@color/gray_f5"
9      android:gravity="center_horizontal">
10
11
12      <TimePicker
13          android:id="@+id/time_picker"
14          android:layout_width="wrap_content"
15          android:layout_height="wrap_content"
16          android:layout_marginTop="30dp"
17          android:timePickerMode="spinner">
18      </TimePicker>
19
20
21      <Button
22          android:id="@+id/btn_confirm"
23          android:layout_width="wrap_content"
24          android:layout_height="wrap_content"
25          android:layout_marginTop="30dp"
26          android:text="确认选择"
27          android:textSize="16sp"
28          android:backgroundTint="@color/blue"/>
29
30
31      <TextView
32          android:id="@+id/tv_selected_time"
33          android:layout_width="wrap_content"
34          android:layout_height="wrap_content"
35          android:layout_marginTop="20dp"
36          android:text="已选择时间：暂无"
37          android:textSize="16sp"
38          android:textColor="@color/black"/>
39
40
41      <TextView
42          android:id="@+id/tv_formatted_time"
```

```
43         android:layout_width="wrap_content"
44         android:layout_height="wrap_content"
45         android:layout_marginTop="10dp"
46         android:text="接口格式：暂无"
47         android:textSize="14sp"
48         android:textColor="@color/gray_99"/>
49
50     </LinearLayout>
```

补充颜色资源（res/values/color.xml），确保样式正常显示：

Code block

```
1     <resources>
2         <color name="white">#FFFFFF</color>
3         <color name="gray_f5">#F5F5F5</color>
4         <color name="gray_e0">#E0E0E0</color>
5         <color name="gray_99">#999999</color>
6         <color name="black">#000000</color>
7         <color name="blue">#2196F3</color>
8         <color name="blue_dark">#1976D2</color>
9         <color name="purple_500">#9C27B0</color>
10    </resources>
```

步骤2：代码中绑定与配置TimePicker

在 `MainActivity.kt` 中绑定控件，设置时间制、默认时间，并通过回调监听时间变化：

Code block

```
1     import android.os.Bundle
2     import android.widget.Button
3     import android.widget.TextView
4     import android.widget.TimePicker
5     import android.widget.Toast
6     import androidx.appcompat.app.AppCompatActivity
7     import java.text.SimpleDateFormat
8     import java.util.Calendar
9     import java.util.Locale
10
11     class MainActivity : AppCompatActivity() {
12         private lateinit var timePicker: TimePicker
13         private lateinit var tvSelectedTime: TextView
14         private lateinit var tvFormattedTime: TextView
15
16         override fun onCreate(savedInstanceState: Bundle?) {
```

```
17     super.onCreate(savedInstanceState)
18     setContentView(R.layout.activity_main)
19
20     // 1. 绑定控件
21     timePicker = findViewById(R.id.time_picker)
22     tvSelectedTime = findViewById(R.id.tv_selected_time)
23     tvFormattedTime = findViewById(R.id.tv_formatted_time)
24     val btnConfirm = findViewById<Button>(R.id.btn_confirm)
25
26     // 2. 配置TimePicker基础属性
27     configTimePicker()
28
29     // 3. 监听时间实时变化 (滑动滚轮时触发)
30     timePicker.setOnTimeChangedListener { _, hour, minute ->
31         updateTimeDisplay(hour, minute, false)
32     }
33
34     // 4. 确认按钮点击事件 (获取最终选择的时间)
35     btnConfirm.setOnClickListener {
36         val selectedHour = TimePickerUtils.getSelectedHour(timePicker)
37         val selectedMinute = TimePickerUtils.getSelectedMinute(timePicker)
38         updateTimeDisplay(selectedHour, selectedMinute, true)
39         Toast.makeText(this, "时间已确认", Toast.LENGTH_SHORT).show()
40     }
41 }
42
43 // 配置TimePicker基础属性
44 private fun configTimePicker() {
45     // 设置时间制: true=24小时制, false=12小时制
46     timePicker.is24HourView = true
47
48     // 设置默认时间为当前时间 (通过工具类封装, 兼容低版本)
49     val calendar = Calendar.getInstance()
50     TimePickerUtils.setDefaultTime(timePicker,
51     calendar.get(Calendar.HOUR_OF_DAY), calendar.get(Calendar.MINUTE))
52 }
53
54 // 更新时间显示 (含普通显示与接口格式)
55 private fun updateTimeDisplay(hour: Int, minute: Int, isConfirmed: Boolean) {
56     {
57         // 1. 普通显示 (区分24小时制与12小时制, 通过工具类转换)
58         val displayInfo = TimePickerUtils.formatDisplayTime(hour, minute,
59         timePicker.is24HourView)
60         val prefix = if (isConfirmed) "已确认时间: " else "当前选择: "
61         tvSelectedTime.text = "$prefix${displayInfo.first}
62         ${displayInfo.second}"
63     }
64 }
```

```
60         // 2. 接口格式 (统一为HH:mm:ss)
61         val formattedTime = TimePickerUtils.formatApiTime(hour, minute)
62         tvFormattedTime.text = "接口格式: $formattedTime"
63     }
64 }
```

步骤3：核心属性与API解析

原生 `TimePicker` 的核心能力通过XML属性和Java/Kotlin API实现，以下是高频用法分类说明：

1. XML布局属性（直接在布局中配置）

属性名	作用	可选值/示例
android:timePickerMode	设置时间选择模式（API 21+）	spinner（滚轮模式）、clock（时钟模式）
android:is24HourView	设置是否为24小时制	true（24小时制）、false（12小时制）
android:textColorPrimary	设置时间文本颜色	@color/black、#000000
android:textSize	设置时间文本大小	16sp、18sp
android:calendarViewShown	是否显示日历视图（仅部分版本支持）	true、false（默认false）

2. Kotlin/Java核心API（代码中动态配置）

API方法/属性	作用	使用示例
is24HourView	获取/设置时间制（Boolean）	timePicker.is24HourView = true
hour（API 23+）	获取/设置选中的小时	timePicker.hour = 14（24小时制）
minute（API 23+）	获取/设置选中的分钟	timePicker.minute = 30
setOnTimeChangeListener	设置时间变化监听器	timePicker.setOnTimeChangeListener { _, h, m -> ... }
setEnabled(Boolean)	设置控件是否可用	timePicker.setEnabled(false)（禁用选择）

getCurrentHour() (API <23)	获取选中的小时（已废弃，用 hour 替代）	timePicker.currentHour = 10
----------------------------	------------------------	-----------------------------

 低版本兼容警告：hour 和 minute 属性在 API 23+ 才公开，API 23 以下需通过反射获取 mCurrentHour 和 mCurrentMinute 字段，否则会抛出 NoSuchFieldException。

三、进阶：自定义 TimePicker 样式（视觉定制）

原生 TimePicker 的默认样式可能与 App 视觉风格冲突，需通过“主题定制”“布局重写”“反射修改内部控件”三种方式实现视觉自定义，满足不同设计需求。

场景1：通过主题修改整体样式（简单高效）

核心思路：在 styles.xml 中定义自定义主题，修改 TimePicker 的文本颜色、选中颜色、背景等属性，再通过布局或代码应用主题。

步骤1：定义自定义主题（res/values/styles.xml）

Code block

```
1  <resources>
2
3      <style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
4          <item name="colorPrimary">@color/blue</item>
5          <item name="colorPrimaryDark">@color/blue_dark</item>
6          <item name="colorAccent">@color/blue</item>
7      </style>
8
9
10     <style name="CustomTimePickerTheme" parent="Theme.AppCompat.Light">
11
12         <item name="android:textColorPrimary">@color/black</item>
13
14         <item name="colorAccent">@color/purple_500</item>
15
16         <item name="android:textColorSecondary">@color/gray_99</item>
17
18         <item name="android:background">@color/white</item>
19
20         <item name="android:textSize">16sp</item>
21     </style>
22 </resources>
```

步骤2：在布局中应用主题

通过 `android:theme` 属性为 `TimePicker` 单独应用自定义主题：

Code block

```
1  <TimePicker
2      android:id="@+id/time_picker"
3      android:layout_width="wrap_content"
4      android:layout_height="wrap_content"
5      android:timePickerMode="spinner"
6      android:theme="@style/CustomTimePickerTheme"/>
```

场景2：重写布局实现完全自定义（复杂样式）

若主题定制无法满足需求（如修改滚轮间距、添加额外装饰），可通过“重写系统布局文件”的方式替换 `TimePicker` 的默认UI结构。核心原理：系统 `TimePicker` 的布局由 `time_picker.xml` 定义，在项目中创建同名布局文件可覆盖系统布局。

步骤1：创建自定义布局文件（res/layout/time_picker.xml）

复制系统 `time_picker.xml` 结构，修改控件样式（如滚轮颜色、间距）：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="wrap_content"
5      android:orientation="vertical"
6      android:padding="16dp">
7
8
9      <NumberPicker
10         android:id="@+id/hour"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:layout_gravity="center_horizontal"
14         android:textColor="@color/purple_500"
15         android:textSize="18sp"
16         android:padding="8dp"/>
17
18
19     <TextView
20         android:layout_width="wrap_content"
21         android:layout_height="wrap_content"
22         android:layout_gravity="center_horizontal"
```

```

23         android:text=":"
24         android:textSize="24sp"
25         android:textColor="@color/purple_500"/>
26
27
28     <NumberPicker
29         android:id="@+id/minute"
30         android:layout_width="wrap_content"
31         android:layout_height="wrap_content"
32         android:layout_gravity="center_horizontal"
33         android:textColor="@color/purple_500"
34         android:textSize="18sp"
35         android:padding="8dp"/>
36
37 </LinearLayout>

```

注：系统 `TimePicker` 的布局结构因Android版本而异，上述布局适用于滚轮模式（spinner），若需适配时钟模式（clock），需参考对应版本的系统布局文件。

步骤3：通过反射关联自定义布局（可选）

若自定义布局修改了控件ID或结构，需通过反射获取内部 `NumberPicker`（小时/分钟滚轮），重新设置数据与监听：

Code block

```

1  // 反射修改TimePicker内部滚轮样式
2  private fun customizeTimePickerWheel(timePicker: TimePicker, theme:
    Resources.Theme) {
3      try {
4          // 1. 获取TimePicker内部的小时和分钟NumberPicker
5          val hourPicker = timePicker.findViewById<android.widget.NumberPicker>
(R.id.hour)
6          val minutePicker = timePicker.findViewById<android.widget.NumberPicker>
(R.id.minute)
7
8          // 2. 自定义滚轮样式（如设置文本颜色、大小）
9          val textColorField =
android.widget.NumberPicker::class.java.getDeclaredField("mSelectorWheelPaint")
10         textColorField.isAccessible = true
11         val paint = textColorField.get(hourPicker) as android.graphics.Paint
12         paint.color = ContextCompat.getColor(this, R.color.purple_500)
13         paint.textSize = resources.getDimension(R.dimen.sp_18)
14
15         // 3. 刷新滚轮显示
16         hourPicker.invalidate()

```



```

17         minutePicker.invalidate()
18     } catch (e: Exception) {
19         e.printStackTrace()
20     }
21 }

```

场景3：修改时间制切换样式（12小时制AM/PM标识）

12小时制下，原生AM/PM标识样式单一，可通过反射修改其文本颜色和大小：

Code block

```

1  // 自定义12小时制的AM/PM标识样式
2  fun customizeAmPmStyle(timePicker: TimePicker, context: Context) {
3      if (timePicker.is24HourView) return
4
5      try {
6          // 1. 获取AM/PM对应的TextView
7          val classForID = Class.forName("com.android.internal.R.$id")
8          val amPmId = classForID.getField("am_pm").get(null) as Int
9          val amPmTextView = timePicker.findViewById<TextView>(amPmId) ?: return
10
11         // 2. 修改样式
12         amPmTextView.apply {
13             textColor = ContextCompat.getColor(context, R.color.purple_500)
14             textSize = 16f
15             setPadding(10.dp2px(context), 0, 10.dp2px(context), 0)
16             typeface = Typeface.DEFAULT_BOLD
17         }
18     } catch (e: Exception) {
19         e.printStackTrace()
20     }
21 }
22
23 // 扩展函数：dp转px
24 fun Int.dp2px(context: Context): Int {
25     return TypedValue.applyDimension(TypedValue.COMPLEX_UNIT_DIP,
26         this.toFloat(), context.resources.displayMetrics).toInt()
27 }

```

四、高级：自定义TimePicker功能（逻辑扩展）

除视觉样式外，实际开发中常需扩展 `TimePicker` 的功能（如时间范围限制、分钟步长控制、禁止选择过去时间），以下是高频场景的完整实现方案。

场景1：限制分钟步长（如只能选择15分钟整数倍）

需求：分钟只能选择00、15、30、45，适配预约时间按刻钟划分的场景。核心思路：通过反射修改 `TimePicker` 内部 `NumberPicker` 的数值范围和显示值。

Code block

```
1  object TimePickerStepUtils {
2      // 设置分钟步长（如15分钟）
3      fun setMinuteStep(timePicker: TimePicker, step: Int, context: Context) {
4          if (step <= 0 || step > 30) {
5              throw IllegalArgumentException("步长需在1-30之间")
6          }
7
8          try {
9              // 1. 获取分钟NumberPicker
10             val classForID = Class.forName("com.android.internal.R$id")
11             val minutePickerId = classForID.getField("minute").get(null) as Int
12             val minutePicker =
13                 timePicker.findViewById<android.widget.NumberPicker>(minutePickerId) ?: return
14
15             // 2. 计算步长对应的数值范围
16             val maxValue = 59 / step // 步长15时, maxValue=3 (0、15、30、45)
17             val displayedValues = Array(maxValue + 1) {
18                 String.format(Locale.CHINA, "%02d", it * step)
19             }
20
21             // 3. 重新配置分钟滚轮
22             minutePicker.apply {
23                 minValue = 0
24                 maxValue = maxValue
25                 this.displayedValues = displayedValues
26                 // 修正默认值：将当前分钟转为最近的步长值
27                 val currentMinute =
28                     TimePickerUtils.getSelectedMinute(timePicker)
29                 value = (currentMinute / step).coerceIn(0, maxValue)
30             }
31
32             // 4. 重写分钟值获取逻辑（将索引转为实际分钟）
33             val minuteField =
34                 TimePicker::class.java.getDeclaredField("mCurrentMinute")
35             minuteField.isAccessible = true
36             val realMinute = minutePicker.value * step
37             minuteField.set(timePicker, realMinute)
38
39             // 5. 监听分钟变化，确保始终为步长整数倍
40             minutePicker.setOnValueChangedListener { _, _, newVal ->
```

```

38         val realMin = newVal * step
39         minuteField.set(timePicker, realMin)
40         // 触发TimePicker的时间变化回调
41         timePicker.setOnTimeChangeListener(null)
42         timePicker.setCurrentMinute(realMin)
43
44     timePicker.setOnTimeChangeListener(timePicker.onTimeChangeListener)
45     }
46     } catch (e: Exception) {
47         e.printStackTrace()
48         Toast.makeText(context, "分钟步长设置失败",
49             Toast.LENGTH_SHORT).show()
50     }
51     // 反射获取TimePicker的时间变化监听器（用于重新设置）
52     private val TimePicker.onTimeChangeListener:
53     TimePicker.OnTimeChangeListener?
54     get() {
55         return try {
56             val field =
57             TimePicker::class.java.getDeclaredField("mOnTimeChangeListener")
58             field.isAccessible = true
59             field.get(this) as TimePicker.OnTimeChangeListener
60         } catch (e: Exception) {
61             null
62         }
63     }
64     // 反射设置分钟值（兼容低版本）
65     private fun TimePicker.setCurrentMinute(minute: Int) {
66         try {
67             if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
68                 this.minute = minute
69             } else {
70                 val field =
71                 TimePicker::class.java.getDeclaredField("mCurrentMinute")
72                 field.isAccessible = true
73                 field.set(this, minute)
74                 // 刷新UI
75                 val method =
76                 TimePicker::class.java.getDeclaredMethod("invalidate")
77                 method.isAccessible = true
78                 method.invoke(this)
79             }
80         } catch (e: Exception) {
81             e.printStackTrace()
82         }
83     }

```

场景2：限制时间范围（如仅允许选择9:00-18:00）

需求：用户只能选择工作时间（9:00-18:00），超出范围时给出提示并自动修正。核心思路：在时间变化回调中校验选中时间，若超出范围则强制设置为边界值。

Code block

```

1  object TimeRangeUtils {
2      // 时间范围限制（最小小时、最大小时）
3      fun limitTimeRange(
4          timePicker: TimePicker,
5          minHour: Int,
6          maxHour: Int,
7          context: Context,
8          onTimeValid: (hour: Int, minute: Int) -> Unit
9      ) {
10         timePicker.setOnTimeChangeListener { _, hour, minute ->
11             var targetHour = hour
12             var targetMinute = minute
13             var isValid = false
14
15             // 1. 校验小时范围
16             when {
17                 hour < minHour -> {
18                     targetHour = minHour
19                     targetMinute = 0
20                     isValid = true
21                 }
22                 hour > maxHour -> {
23                     targetHour = maxHour
24                     targetMinute = 0
25                     isValid = true
26                 }
27                 // 2. 若小时为边界值，校验分钟（如18:00后不允许选择）
28                 hour == maxHour && minute > 0 -> {
29                     targetMinute = 0
30                     isValid = true
31                 }
32             }
33
34             // 3. 超出范围给出提示
35             if (!isValid) {
36                 Toast.makeText(context, "请选择${minHour}:00-${maxHour}:00之间的时
37                 间", Toast.LENGTH_SHORT).show()
38                 // 修正时间（避免递归调用，先移除监听再设置）
39                 updateTimeWithListener(timePicker, targetHour, targetMinute) {

```

```

39             limitTimeRange(timePicker, minHour, maxHour, context,
onTimeValid)
40         }
41     }
42
43     // 4. 时间有效时回调
44     onTimeValid(targetHour, targetMinute)
45 }
46 }
47
48 // 修正时间并重新设置监听 (避免递归)
49 private fun updateTimeWithListener(
50     timePicker: TimePicker,
51     hour: Int,
52     minute: Int,
53     listener: (TimePicker) -> Unit
54 ) {
55     timePicker.setOnTimeChangeListener(null)
56     TimePickerUtils.setDefaultTime(timePicker, hour, minute)
57     listener.invoke(timePicker)
58 }
59 }

```

场景3：禁止选择过去的时间（结合当前时间校验）

需求：用户只能选择当前时间之后的时间，避免选择已过期的时间。核心思路：获取当前时间，与选中时间对比，超出范围则修正。

Code block

```

1  object TimeValidUtils {
2      // 禁止选择过去的时间
3      fun disablePastTime(
4          timePicker: TimePicker,
5          context: Context,
6          onTimeValid: (hour: Int, minute: Int) -> Unit
7      ) {
8          val currentCalendar = Calendar.getInstance()
9          val currentHour = currentCalendar.get(Calendar.HOUR_OF_DAY)
10         val currentMinute = currentCalendar.get(Calendar.MINUTE)
11
12         timePicker.setOnTimeChangeListener { _, hour, minute ->
13             var targetHour = hour
14             var targetMinute = minute
15             var isPast = false
16

```

```

17         // 1. 对比当前时间
18         when {
19             hour < currentHour -> {
20                 targetHour = currentHour
21                 targetMinute = currentMinute
22                 isPast = true
23             }
24             hour == currentHour && minute < currentMinute -> {
25                 targetMinute = currentMinute
26                 isPast = true
27             }
28         }
29
30         // 2. 过去时间给出提示并修正
31         if (isPast) {
32             Toast.makeText(context, "不能选择过去的时间",
33 Toast.LENGTH_SHORT).show()
34             TimeRangeUtils.updateTimeWithListener(timePicker, targetHour,
35 targetMinute) {
36                 disablePastTime(it, context, onTimeValid)
37             }
38         }
39
40         // 3. 回调有效时间
41         onTimeValid(targetHour, targetMinute)
42     }
43
44     // 校验时间是否为过去时间（用于确认按钮二次校验）
45     fun isPastTime(hour: Int, minute: Int): Boolean {
46         val currentCalendar = Calendar.getInstance()
47         val targetCalendar = Calendar.getInstance().apply {
48             set(Calendar.HOUR_OF_DAY, hour)
49             set(Calendar.MINUTE, minute)
50             set(Calendar.SECOND, 0)
51         }
52         // 允许当前分钟的时间（如当前10:05，选择10:05视为有效）
53         targetCalendar.add(Calendar.SECOND, -1)
54         return targetCalendar.before(currentCalendar)
55     }

```

场景4：与DatePicker联动实现“日期+时间”选择

需求：结合 `DatePicker` 选择日期后，`TimePicker` 的时间范围随日期变化（如选择“今天”则禁止过去时间，选择“明天”则无此限制）。

Code block

```
1  class DateTimeLinkManager(  
2      private val timePicker: TimePicker,  
3      private val context: Context,  
4      private val onValidDateTime: (date: Calendar, hour: Int, minute: Int) ->  
Unit  
5  ) {  
6      // 存储选中的日期  
7      private var selectedDate: Calendar = Calendar.getInstance()  
8  
9      // 初始化DatePicker的选择回调  
10     fun initDatePickerCallback(datePicker: DatePicker) {  
11         datePicker.init(  
12             selectedDate.get(Calendar.YEAR),  
13             selectedDate.get(Calendar.MONTH),  
14             selectedDate.get(Calendar.DAY_OF_MONTH)  
15         ) { _, year, month, dayOfMonth ->  
16             selectedDate = Calendar.getInstance().apply {  
17                 set(year, month, dayOfMonth)  
18             }  
19             // 根据选中日期更新时间Picker限制  
20             updateTimeRangeByDate()  
21         }  
22     }  
23  
24     // 根据选中日期更新时间范围  
25     private fun updateTimeRangeByDate() {  
26         val currentCalendar = Calendar.getInstance()  
27         val isToday = selectedDate.isSameDay(currentCalendar)  
28  
29         if (isToday) {  
30             // 选中今天, 禁止过去时间  
31             TimeValidUtils.disablePastTime(timePicker, context) { hour, minute  
->  
32                 onValidDateTime(selectedDate, hour, minute)  
33             }  
34         } else {  
35             // 选中其他日期, 仅限制工作时间 (9:00-18:00)  
36             TimeRangeUtils.limitTimeRange(timePicker, 9, 18, context) { hour,  
minute ->  
37                 onValidDateTime(selectedDate, hour, minute)  
38             }  
39         }  
40     }  
41  
42     // 扩展函数: 判断两个Calendar是否为同一天
```

```

43     private fun Calendar.isSameDay(other: Calendar): Boolean {
44         return this.get(Calendar.YEAR) == other.get(Calendar.YEAR)
45             && this.get(Calendar.MONTH) == other.get(Calendar.MONTH)
46             && this.get(Calendar.DAY_OF_MONTH) ==
other.get(Calendar.DAY_OF_MONTH)
47     }
48 }
49
50 // 使用方式
51 val datePicker = findViewById<DatePicker>(R.id.date_picker)
52 val dateTimeLinkManager = DateTimeLinkManager(timePicker, this) { date, hour,
minute ->
53     // 处理有效的日期+时间
54     val dateStr = SimpleDateFormat("yyyy-MM-dd",
Locale.CHINA).format(date.time)
55     val timeStr = SimpleDateFormat("HH:mm",
Locale.CHINA).format(Calendar.getInstance().apply {
56         set(Calendar.HOUR_OF_DAY, hour)
57         set(Calendar.MINUTE, minute)
58     }.time)
59     tvDateTime.text = "选中: $dateStr $timeStr"
60 }
61 dateTimeLinkManager.initDatePickerCallback(datePicker)

```

五、实战优化：工具类封装与性能考量

在实际开发中，为提高代码复用性和可维护性，需将 `TimePicker` 的通用逻辑（如低版本兼容、时间格式化、步长设置）封装为工具类，并关注性能优化点。

1. 核心工具类封装（TimePickerUtils.kt）

Code block

```

1  object TimePickerUtils {
2      /**
3       * 获取选中的小时（兼容所有API版本）
4       */
5      fun getSelectedHour(timePicker: TimePicker): Int {
6          return if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
7              timePicker.hour
8          } else {
9              getFieldValue(timePicker, "mCurrentHour") ?: 0
10         }
11     }
12 }

```



```

13  /**
14   * 获取选中的分钟（兼容所有API版本）
15   */
16  fun getSelectedMinute(timePicker: TimePicker): Int {
17      return if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
18          timePicker.minute
19      } else {
20          getFieldValue(timePicker, "mCurrentMinute") ?: 0
21      }
22  }
23
24  /**
25   * 设置默认时间（兼容所有API版本）
26   */
27  fun setDefaultTime(timePicker: TimePicker, hour: Int, minute: Int) {
28      if (hour < 0 || hour > 23 || minute < 0 || minute > 59) {
29          throw IllegalArgumentException("时间参数不合法")
30      }
31
32      if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
33          timePicker.hour = hour
34          timePicker.minute = minute
35      } else {
36          setFieldValue(timePicker, "mCurrentHour", hour)
37          setFieldValue(timePicker, "mCurrentMinute", minute)
38          // 低版本需手动刷新UI
39          timePicker.invalidate()
40      }
41  }
42
43  /**
44   * 格式化显示时间（区分12/24小时制）
45   * @return Pair(时段描述, 时间字符串) 如（上午, 10:30）
46   */
47  fun formatDisplayTime(hour: Int, minute: Int, is24Hour: Boolean):
Pair<String, String> {
48      val displayHour = if (is24Hour) {
49          hour
50      } else {
51          when (hour) {
52              0 -> 12
53              in 1..12 -> hour
54              else -> hour - 12
55          }
56      }
57      val period = when {
58          is24Hour -> ""

```

```

59         hour < 6 -> "凌晨"
60         hour < 12 -> "上午"
61         hour == 12 -> "中午"
62         hour < 18 -> "下午"
63         else -> "晚上"
64     }
65     val timeStr = String.format(Locale.CHINA, "%02d:%02d", displayHour,
minute)
66     return Pair(period, timeStr)
67 }
68
69 /**
70  * 格式化接口提交时间 (HH:mm:ss)
71  */
72 fun formatApiTime(hour: Int, minute: Int): String {
73     val calendar = Calendar.getInstance().apply {
74         set(Calendar.HOUR_OF_DAY, hour)
75         set(Calendar.MINUTE, minute)
76         set(Calendar.SECOND, 0)
77     }
78     return SimpleDateFormat("HH:mm:ss", Locale.CHINA).format(calendar.time)
79 }
80
81 /**
82  * 反射获取字段值
83  */
84 private fun getFieldValue(obj: Any, fieldName: String): Int? {
85     return try {
86         val field = obj.javaClass.getDeclaredField(fieldName)

```

2. 性能优化关键点

- **减少反射调用次数**：反射性能开销较大，将反射逻辑封装为工具类，避免在循环或频繁回调中调用。可通过静态变量缓存反射获取的Field对象，进一步优化性能。
- **避免递归回调**：在时间范围限制逻辑中，修正时间前需先移除 `OnTimeChangeListener`，设置完成后重新绑定，防止触发递归调用导致ANR。
- **合理使用 `invalidate()` 刷新UI**：低版本中修改时间后需手动调用 `invalidate()` 刷新UI，但高版本无需额外操作，避免不必要的UI重绘。
- **横竖屏切换数据保存**：在Activity/Fragment的 `onSaveInstanceState` 中保存选中的时间，在 `onRestoreInstanceState` 中恢复，避免横竖屏切换后数据丢失。

Code block

```

1  // 横竖屏切换保存时间数据
2  override fun onSaveInstanceState(outState: Bundle) {
3      super.onSaveInstanceState(outState)

```

```

4     val hour = TimePickerUtils.getSelectedHour(timePicker)
5     val minute = TimePickerUtils.getSelectedMinute(timePicker)
6     outState.putInt("SELECTED_HOUR", hour)
7     outState.putInt("SELECTED_MINUTE", minute)
8 }
9
10 // 恢复时间数据
11 override fun onRestoreInstanceState(savedInstanceState: Bundle) {
12     super.onRestoreInstanceState(savedInstanceState)
13     val hour = savedInstanceState.getInt("SELECTED_HOUR")
14     val minute = savedInstanceState.getInt("SELECTED_MINUTE")
15     TimePickerUtils.setDefaultTime(timePicker, hour, minute)
16     updateTimeDisplay(hour, minute, false)
17 }

```

3. 兼容性处理技巧

- **API版本判断**: 使用 `Build.VERSION.SDK_INT` 区分高/低版本API, 避免调用不存在的方法或属性。
- **主题兼容**: 确保自定义主题继承自 `Theme.AppCompat` 系列主题, 避免在低版本设备上出现 `InflateException`。
- **屏幕适配**: 滚轮模式下, 通过 `dp2px` 工具类转换尺寸单位, 避免不同屏幕密度下样式错乱; 时钟模式下, 可通过 `android:scaleX` 和 `android:scaleY` 调整大小。

六、特殊场景适配与问题排查

1. 沉浸式状态栏下的布局偏移

问题: 当Activity使用沉浸式状态栏时, `TimePicker` 可能会向上偏移, 与状态栏重叠。

解决方案: 在布局中为 `TimePicker` 添加顶部内边距, 适配状态栏高度:

Code block

```

1 // 为TimePicker设置顶部内边距 (适配沉浸式状态栏)
2 fun adjustTimePickerPadding(timePicker: TimePicker, context: Context) {
3     val resourceId = context.resources.getIdentifier("status_bar_height",
4         "dimen", "android")
5     if (resourceId > 0) {
6         val statusBarHeight =
7             context.resources.getDimensionPixelSize(resourceId)
8         timePicker.setPadding(0, statusBarHeight + 20.dp2px(context), 0, 0)
9     }
10 }

```

2. 部分机型滚轮样式异常（如小米、华为）

问题：部分厂商对系统控件进行了定制，导致 `TimePicker` 的滚轮样式与预期不符（如颜色、间距异常）。

解决方案：1. 优先使用主题定制，避免依赖系统默认样式；2. 若主题无效，采用重写布局的方式，完全自定义滚轮样式；3. 测试时覆盖主流机型，针对性处理适配问题。

3. 时间选择与服务器时区同步问题

问题：本地时间选择后提交到服务器，因时区差异导致时间错误（如本地时区为东八区，服务器为UTC时区）。

解决方案：提交服务器时将本地时间转换为UTC时间，获取服务器时间时再转换为本地时间：

Code block

```
1  // 本地时间转UTC时间（用于提交服务器）
2  fun localTimeToUtc(hour: Int, minute: Int): String {
3      val localCalendar = Calendar.getInstance()
4      localCalendar.set(Calendar.HOUR_OF_DAY, hour)
5      localCalendar.set(Calendar.MINUTE, minute)
6      localCalendar.set(Calendar.SECOND, 0)
7
8      val utcCalendar = Calendar.getInstance(TimeZone.getTimeZone("UTC"))
9      utcCalendar.timeInMillis = localCalendar.timeInMillis
10
11     return SimpleDateFormat("HH:mm:ss", Locale.CHINA).format(utcCalendar.time)
12 }
13
14 // UTC时间转本地时间（用于显示服务器返回时间）
15 fun utcToLocalTime(utcTime: String): String {
16     val sdf = SimpleDateFormat("HH:mm:ss", Locale.CHINA)
17     sdf.timeZone = TimeZone.getTimeZone("UTC")
18     val utcDate = sdf.parse(utcTime) ?: return utcTime
19
20     sdf.timeZone = TimeZone.getDefault()
21     return sdf.format(utcDate)
22 }
```

七、总结与扩展推荐

`TimePicker` 作为Android原生时间选择控件，具备轻量、适配性强的特点，其使用需根据业务场景选择合适的定制方案：简单样式需求用“主题定制”，复杂UI需求用“布局重写+反射”，功能扩展需

求则通过“时间监听+逻辑校验”实现。开发中需重点关注低版本兼容（尤其是API 23以下的反射处理）、时间制转换逻辑及性能优化，避免出现数值异常或ANR问题。

扩展推荐：

- **Material Design风格选择器**：Google官方Material库中的 `MaterialTimePicker`，支持Material Design 3样式，内置多种主题，适配性更强，推荐AndroidX项目使用。
- **开源选择器库**：如 `PickerView`（支持时间、日期、地址等多种选择，高度可定制）、`Android-PickerView`（轻量级封装，支持分钟步长、时间范围限制）、`WheelPicker`（纯自定义滚轮控件，样式统一）。
- **国际化适配**：通过 `Locale` 类控制时间显示格式，如英文环境下显示“10:30 AM”，中文环境下显示“上午 10:30”；同时适配不同地区的时间制习惯（如欧美常用12小时制，中国常用24小时制）。
- **Jetpack Compose适配**：若使用Jetpack Compose开发，可使用Compose原生的 `TimePicker` 组件，其API更简洁，支持compose风格的自定义。

本文覆盖了 `TimePicker` 从基础到高级的全场景使用需求，开发者可根据实际业务提取工具类和管理类，封装通用逻辑，提高开发效率。在实际项目中，需结合产品需求、目标设备范围及技术栈选择最合适的实现方案，确保时间选择功能的稳定性和用户体验。

（注：文档部分内容可能由 AI 生成）